# Building a CNN for Rock-Paper-Scissors Classification

## Index


1. **Introduction**
2. **Data Exploration and Augmentation**
   - 2.1 Dataset exploration and preprocessing
   - 2.2 Data Augmentation
3. **CNN Architectures**
   - 3.1 General Training Set-up
   - 3.2 Model 1: Baseline CNN
   - 3.3 Model 2: Deep Unregularized CNN
   - 3.4 Model 3: Tuned and Regularized CNN
4. **Evaluation and Analysis**
   - 4.1 Training Dynamics
     - 4.1.1 Model 1 Performance
     - 4.1.2 Model 2 Performance
     - 4.1.3 Model 3 Performance
   - 4.2 Test Set Evaluation
5. **Generalization Test**
   - 5.1 Setup and Methodology
   - 5.2 Findings and Analysis
6. **Conclusions**

# 1. Introduction

The objective of this project is to design, train, and evaluate Deep Learning models capable of accurately classifying images of hands performing the “rock”, “paper”, and “scissors” gestures. Convolutional Neural Networks (CNNs) were selected for this task, as they are the foundational standard for image classification problems.

The images used for training and evaluation come from a publicly available Kaggle dataset (https://www.kaggle.com/datasets/drgfreeman/rockpaperscissors). The dataset consists of hand gesture images captured in an identical context: the hand is always framed from the same side against a uniform green background. While this clean environment accelerates the training process by making the subject easy to isolate, it amplifies the risk of the model overfitting by memorizing the background context rather than the true shape of the gesture. To mitigate this risk, robust data augmentation techniques are introduced during the preprocessing pipeline.

To thoroughly explore this classification task, three models of increasing complexity are designed, trained, and evaluated. The first model is a shallow baseline architecture, fast to train and implement. The following one is a deep, highly parameterized model designed to test the network's raw capacity limits. The last one is an optimized architecture that matches the depth of the second model but introduces regularization (Dropout) and automated hyperparameter tuning via the Hyperband algorithm.

Finally, the performance of the models will be evaluated on a strictly held-out test set using multiple classification metrics. Furthermore, a test using out-of-distribution (personal) images will be conducted to evaluate how effectively the models adapt to real-world images with complex, non-uniform backgrounds.

# 2. Data Exploration and Augmentation

## 2.1 Dataset exploration and preprocessing

The dataset consists of 2,188 images that are almost perfectly balanced across the three classes (approximately 720 images per class), as shown in *Figure 1*. This even distribution is crucial, as it prevents the neural network from developing a predictive bias toward a majority class during training.

*Figure 1*: Distribution of classes in the dataset

To ensure rigorous training and evaluation, the dataset was partitioned using the `splitfolders` library into three distinct subsets: training set (70% - 1531 images), validation set (15% - 326 images), and test set (15% - 331 images). The training set is used exclusively to update the models' weights, the validation set is used to monitor performance and tune hyperparameters, and the test set is reserved strictly for the final, unbiased evaluation phase.

All raw images in the dataset share a rectangular resolution of 300x200 pixels. To satisfy the fixed-input requirements of Convolutional Neural Networks, all images were resized to 150x150 square pixels. This dimension provides an optimal balance between preserving critical geometric details of the hands and minimizing computational overhead. Additionally, pixel intensities were normalized by rescaling them from a [0, 255] integer range to a [0, 1] float range. This standardization ensures faster and more stable convergence during gradient descent.

## 2.2 Data Augmentation

The images in the dataset share a nearly identical context. In particular, the hands consistently enter the frame from the same side against a uniform green background. While this clean environment is helpful, it severely amplifies the risk of the models overfitting: the models may memorize the static background or positional context rather than the true shape of the hand gestures.

To mitigate this risk, a data augmentation pipeline was implemented using Keras' `ImageDataGenerator`. It is important to note that these transformations were applied

exclusively to the training data. Each training image is subjected to random transformations, including:

i. Rotation: Up to 40° left or right.

ii. Shifting: Moved off-center vertically or horizontally by up to 20%.

iii. Zooming & Shearing: Up to 20% to simulate different viewing distances and perspective angles.

iv. Flipping: Horizontally flipped 50% of the time to ensure the network recognizes both left and right-hand orientations.

*Figure 2* illustrates how this data augmentation pipeline alters a standard training image, forcing the network to learn features rather than memorizing exact pixel locations. Notice that the pipeline also fills any empty pixels created by rotation/movement with the color of the nearest valid pixel, creating those streaks at the edges.

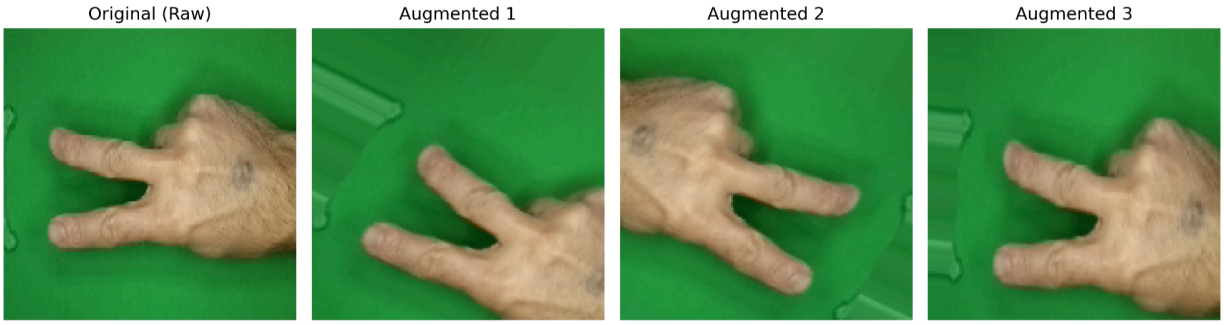


*Figure 2:* Data Augmentation on a "Scissors"-labeled image

# 3. CNN Architectures

## 3.1 General Training Set-up

All implemented models were compiled using the Adam optimizer (with an initial learning rate of 0.001) and the categorical cross-entropy loss function, which is the standard for multi-class classification tasks. Training was configured for a maximum of 25 epochs, processing the data in batch sizes of 32 images.

To optimize computational efficiency and ensure stable convergence, two dynamic callbacks were integrated into all training pipelines:

- `EarlyStopping`: monitored the validation loss to prevent over-training. If the validation loss failed to improve for 5 consecutive epochs, training was halted, and the model weights from the best-performing epoch were automatically restored.
- `ReduceLROnPlateau`: dynamically reduced the learning rate by a factor of 0.2 if the validation loss plateaued for 3 epochs. This learning rate decay allows the optimizer to take smaller, more precise steps toward the loss minimum during the later stages of training.

## 3.2 Model 1: Baseline CNN

The first model establishes a shallow baseline architecture. Despite its structural simplicity, we hypothesized it would perform quite well because the gestures stand out clearly against the clean, uniform backgrounds of the dataset images.

The feature extraction base consists of just two Convolutional blocks: the first utilizes 16 filters, and the second utilizes 32 filters. Both employ ReLU activation functions and are followed by 2x2 Max Pooling layers to progressively downsample the spatial dimensions. The resulting feature map is flattened and passed into a single Dense classifier layer of 64 neurons (ReLU activation). The network concludes with a 3-neuron output layer utilizing a softmax activation function, which is strictly required to convert the final outputs into a normalized probability distribution for the Rock, Paper, and Scissors classes.

## 3.3 Model 2: Deep Unregularized CNN

The second architecture significantly increases the structural complexity and parameter count by utilizing a much deeper network. It consists of four Convolutional blocks (featuring 32, 64, 128, and 256 filters, respectively), all utilizing ReLU activation functions and followed by 2x2 Max Pooling layers to progressively downsample the spatial dimensions. The resulting feature map is flattened and passed into a big, single Dense classifier layer of 512 neurons (ReLU activation). The network concludes with the standard 3-neuron softmax output layer. We expect this model to overfit the training data given its over-parameterized nature and complete lack of explicit regularization.

## 3.4 Model 3: Tuned and Regularized CNN

The final model maintains the deep 4-Convolutional block structure of Model 2, but introduces algorithmic complexity. Most importantly, a rigorous regularization strategy

is introduced: Dropout. Dropout layers are inserted after each Convolutional block and before the final classification layer to randomly deactivate a percentage of neurons during training. This should help to prevent the network from memorizing the static green background of the training data.

Furthermore, automated hyperparameter tuning is introduced to mathematically optimize the network. The optimal learning rate, the two ideal dropout rates (one for the convolutional bases and one for the dense head), and the optimal number of dense units were searched using the Hyperband algorithm (via `KerasTuner`). Unlike a standard Grid Search, which trains every possible combination to completion (being computationally expensive), or a Random Search, which randomly samples configurations but trains them all equally regardless of their early performance, Hyperband treats the tuning process as a resource allocation problem. The algorithm uses a "tournament" bracket system to dynamically allocate training epochs. It starts with many random configurations running for just a few epochs, immediately discards the poor performers, and promotes only the top models to the next round with increased training time. Through this rigorous search, the Hyperband algorithm identified the following optimal configuration: 384 Dense units, a learning rate of 0.001, a spatial dropout rate of 0.4, and a final dropout rate of 0.5.

# 4. Evaluation and Analysis

## 4.1 Training Dynamics

To understand how each model actually learned, we plotted the training and validation accuracy and loss curves for all three models across their training epochs. This allows us to understand whether the model under or overfit on the data provided, analysing whether the changes made in the pipelines (both in data, with data augmentation, and in model structure, drop-out deeper model..) had an impact in the training process. In the following analysis we will take into consideration accuracy only, since the loss values are pretty similar in all cases.

### 4.1.1 Model 1 Performance

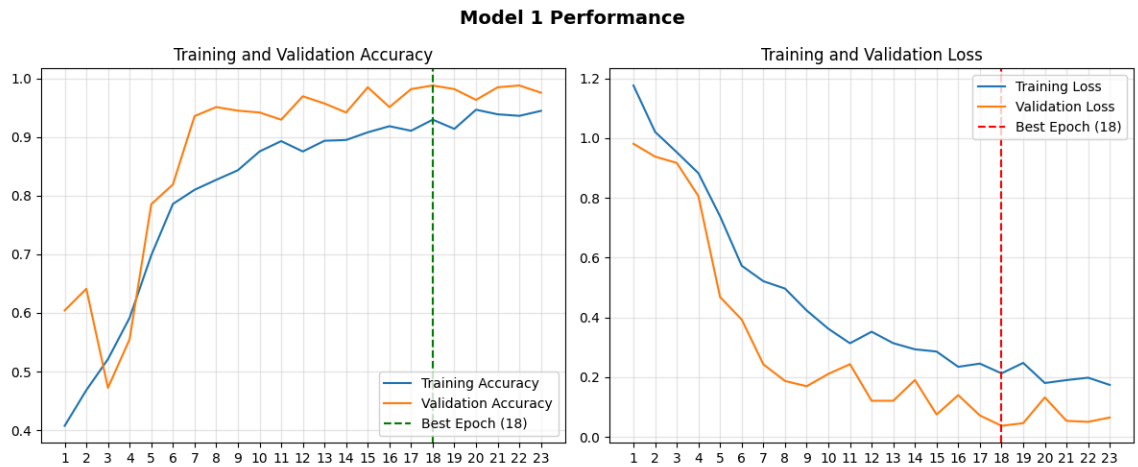


*Figure 3:* Model 1 (Baseline) Performance

Model 1 plots are reported in *Figure 3*. The baseline model trained following a smooth and stable pattern. Thanks to the heavy data augmentation applied to the training set (see 2.2) the training accuracy was constrained climbing from 40% to 94% roughly. Conversely, the validation accuracy evaluated on clean images quickly accelerated often exceeding the training accuracy, which strongly indicates a healthy fit with no initial overfitting. At epoch 21 the `ReduceLROnPlateau` callback detected a flattening in the validation loss and successfully reduced the learning rate from 0.001 to 0.0002. Still, the `EarlyStopping` callback stopped training at Epoch 23 after the validation loss failed to improve further, automatically restoring the model's optimal weights from Epoch 18. In conclusion the model achieved the exceptional validation accuracy of 98.77%.

## 4.1.2 Model 2 Performance

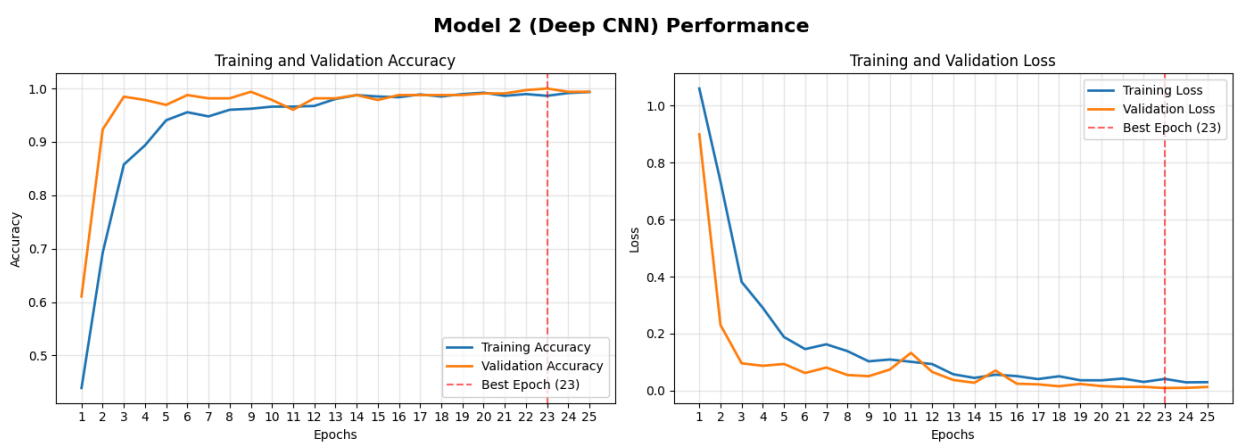


*Figure 4:* Model 2 (Deep CNN) Performance

Analyzing the training and validation metrics of the Deep CNN (*Figure 4*) reveals that our initial hypothesis was partially incorrect. We originally anticipated that this massive, unregularized model might overfit the training data. However, the learning curves demonstrate rapid convergence without classic overfitting indicators. This stability is likely attributable to the aggressive data augmentation applied to the training set, which successfully prevented the network from memorizing exact spatial pixel locations. By Epoch 12, the `ReduceLROnPlateau` callback appropriately reduced the learning rate, enabling the optimizer to make precise micro-adjustments. Ultimately, at Epoch 23, the model achieved a flawless validation accuracy of 100% and an exceptionally low validation loss of 0.0090.

While these results appear phenomenal, achieving a perfect score on an in-distribution validation set warrants critical caution. This flawless performance suggests that the model might have exploited static environmental conditions, like the uniform green background and consistent lighting, that were not altered by our data augmentation. Consequently, while Model 2 acts as a perfect in-distribution classifier, its lack of regularization means it may fail when tested in different contexts.

### 4.1.3 Model 3 Performance

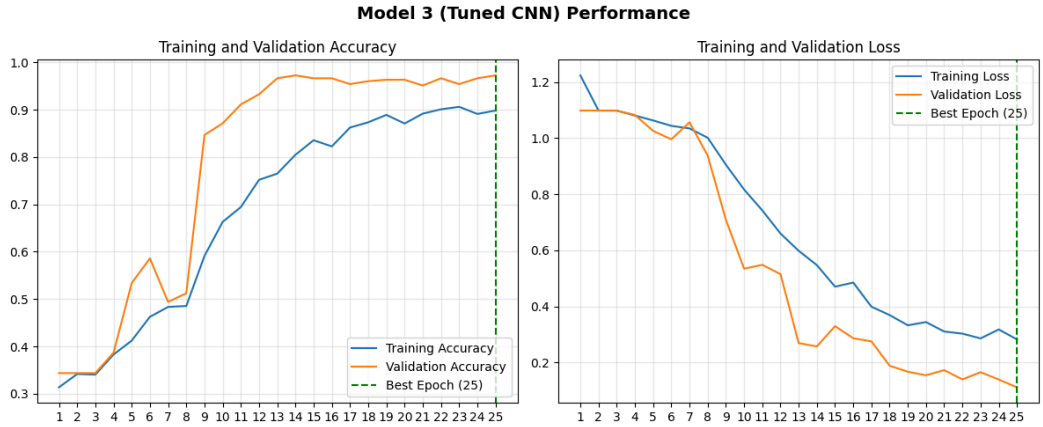


*Figure 5:* Model 3 (Tuned CNN) Performance

The training dynamics of the Tuned and Regularized CNN (Figure 5) exhibit a deliberately slower and more restrained convergence compared to Model 2. For the first 8 epochs, the training accuracy struggled to surpass 50%. This restrained initial progress is the direct, intended consequence of applying Dropout regularization. By randomly deactivating a percentage of neurons during training, we aim at preventing

the network from relying less on the context of the images, and more on the actual hand gesture.

At Epoch 17, the `ReduceLROnPlateau` callback was activated, lowering the learning rate to 0.0002 and allowing the model to smoothly fine-tune its weights. Ultimately, the model successfully completed its full 25-epoch cycle without triggering `EarlyStopping`. It finished with a training accuracy of 89.8% and a highly stable validation accuracy of 97.24%.

## 4.2 Test Set Evaluation

To evaluate the true performance of our architectures, all three models were tested on a strictly held-out test set consisting of 331 images. It is important to note that this test set comes from the same source of the training data (the Kaggle dataset described in the beginning) and all the images share the same environmental features already discussed.

*Figures 6, 7* and *8* portray the Confusion Matrices of all three models.

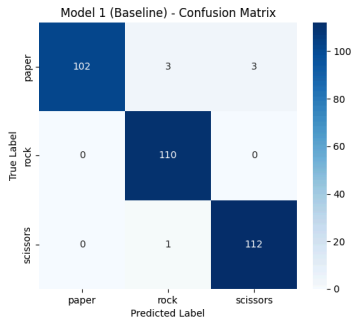


*Figure 6*: Model 1 - Confusion Matrix

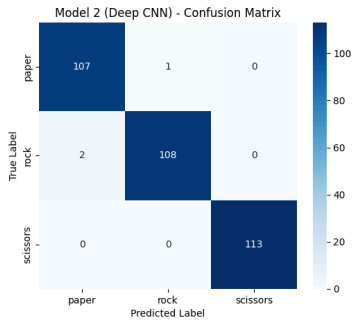


*Figure 7:* Model 2 - Confusion Matrix

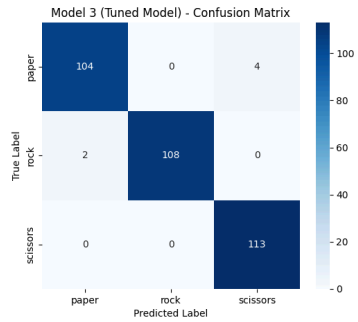


*Figure 8:* Model 3 - Confusion Matrix

All models perform really well with tiny differences.

1. The baseline architecture achieved an overall accuracy of 98%. Looking at the specific class metrics, it achieved a precision of 1.00 for the 'paper' class but a recall of 0.94. This indicates that while the model was highly cautious and accurate when it predicted 'paper', it missed roughly 6% of the actual 'paper' images in the test set. Looking at the actual misclassified images by Model 1 (*Figure 9*) we realise that the wrongfully predicted paper instances either show more wide spread fingers

(classified as scissors) or tighter fingers (classified as rock), showing that the Model 1 struggles with such paper gestures.

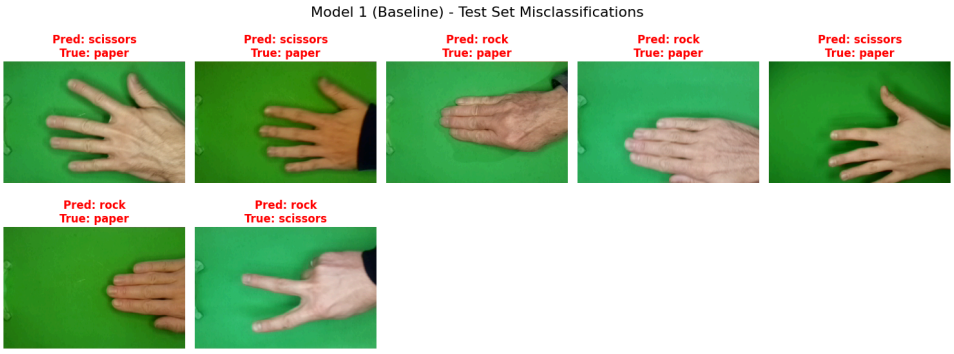


*Figure 9:* Model 1 - Misclassified images

2. The Deep CNN model is the clear winner on the images of the dataset, achieving a near perfect accuracy of 99%. This performance aligns perfectly with our training dynamics analysis: because the model has massive parameter capacity (512 Dense units) and lacks regularization, we hypothesize it was able to map the clean, uniform features of the dataset's green-screen environment.

3. The Tuned Model achieved a 98% overall accuracy, matching the baseline Model 1 but slightly underperforming the unregularized Model 2. This marginal decrease in in-distribution accuracy represents the intended mathematical trade-off of introducing aggressive Dropout regularization. Similar to the baseline model, Model 3 exhibited a slight comparative struggle with the 'paper' class, yielding a 0.97 F1-Score. An analysis of the four misclassified instances (*Figure 10*) reveals a pattern consistent with the conclusions drawn for Model 1: the model occasionally struggles to correctly classify edge cases, such as confusing 'paper' and 'scissors' when the fingers are spread unusually wide or captured at severe angles.

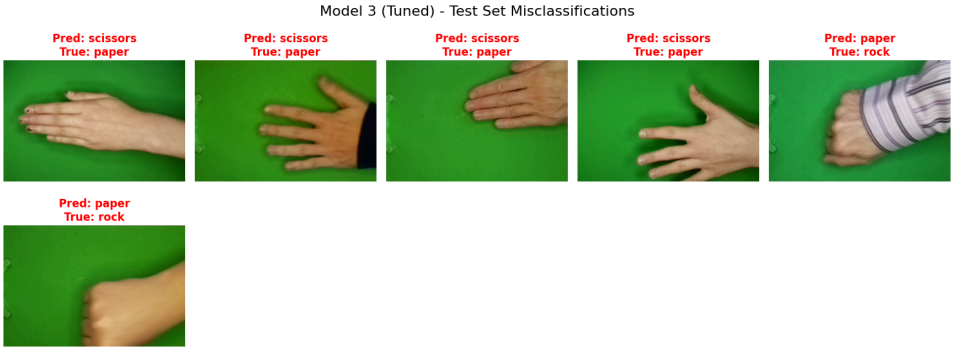


*Figure 10:* Model 3 - Misclassified images

If the final objective of this project were strictly to optimize performance on images sharing the identical environmental context of the training dataset, Model 2 would be the optimal choice. However, because Model 2 lacks explicit regularization and relies on a massive number of parameters, it is highly prone to environmental overfitting; it is highly likely to fail on out-of-distribution images. Conversely, Model 3 deliberately sacrificed a fraction of in-distribution accuracy to ensure structural robustness.

To definitively test this hypothesis and evaluate the true real-world viability of these architectures, the next and final section of this report subjects all three models to a Generalization Test using out-of-distribution (personal) images.

# 5. Generalization test

## 5.1 Setup and Methodology

To test the generalization capabilities of the models, a custom dataset of 42 personal images was captured, perfectly balanced across the "Rock," "Paper," and "Scissors" classes. These images were captured in different environmental conditions from those of the Kaggle dataset used till now. They were taken against a variety of backgrounds (white wall, black surfaces, wood grain,..) and in different lighting conditions affecting the shadows of the hand gestures. The idea is that testing on this dataset will reveal whether the models actually learned the geometric structure of human hands, or were highly helped by the original environment.

## 5.2 Findings and Analysis

As expected, all models experienced a significant drop in accuracy when exposed to the out-of-distribution data, but the variance between their performances is interesting to notice:

1. Model 1 (Baseline): the shallow baseline model managed a 45.2% accuracy. It showed a sharp decline from its 98% in-distribution score, still it performed better than random chance. Possibly its structural simplicity prevented it from fully memorizing the green background, allowing it to retain some basic detection capabilities.
2. Model 2 (Deep Unregularized): the deep, unregularized model suffered a catastrophic failure. Dropping from a near-perfect 99% on the test set down to

33.3%, its performance degenerated to random guessing. This proves our earlier hypothesis: Model 2 suffered from environmental overfitting. It used its massive 512-neuron capacity to perfectly map the  green background and lighting of the training set, failing entirely to learn the invariant shapes of the hand gestures.

3. Model 3 (Tuned and Regularized): the champion model outperformed the others, achieving 57.1% accuracy. While the domain shift still heavily impacted its performance, this result represents a victory for algorithmic regularization. The aggressive Dropout implemented during training successfully forced the network to learn more robust and generalized features rather than relying entirely on the background context.

In conclusion, while Model 2 appeared superior in a controlled environment, its lack of regularization made it useless in the real world. Model 3, by trading a fraction of training accuracy for algorithmic robustness (Dropout and Hyperband tuning), emerged as the only architecture capable of meaningfully adapting to different environments.

# 6. Conclusions

This project successfully designed, trained, and evaluated three Convolutional Neural Networks (CNNs) of increasing complexity to classify images of "rock," "paper," and "scissors" hand gestures. Because the training images shared a highly uniform environmental background, a robust data augmentation pipeline was implemented to prevent spatial overfitting. The baseline architecture (Model 1) established a strong foundation, achieving a 98% accuracy on the in-distribution test set. Model 2, a highly parameterized and unregularized deep CNN, leveraged its massive capacity to perfectly map the uniform dataset, achieving a near-flawless 99% test accuracy. Model 3 shared the deep architecture of Model 2 but introduced rigorous Dropout regularization and hyperparameter tuning via the Hyperband algorithm, maintaining an excellent 98% in-distribution accuracy. However, the Generalization Test on out-of-distribution (OOD) personal images revealed the true, underlying behavior of these architectures. The heavily parameterized Model 2 failed miserably, dropping to a random-chance accuracy of 33.3%. Conversely, Model 3 proved to be the only architecture capable of genuine generalization, surviving the domain shift and achieving 57.1% accuracy. This definitively highlights the danger of relying exclusively on in-distribution metrics and proves the mathematical necessity of algorithmic regularization for real-world deployment.

While Model 3 successfully demonstrated the mechanics of generalization, we acknowledge that an absolute OOD accuracy of 57.1% remains suboptimal for practical applications. To address this limitation in future developments, the training dataset must be heavily diversified with varied backgrounds, dynamic lighting, and different skin tones. Additionally, future iterations could implement an automated background removal preprocessing step to isolate the hand and eliminate environmental noise before the image ever reaches the neural network.

---

*I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.*

---